Part 2：OpenClaw 部署环境对比

> *四大平台各有所长，没有"不能用"的平台，只有"最适合"的平台。这一章用数据帮你做判断。*

引言

你可能听过这样的说法："Linux 才是最稳定的""Mac 体验最好""Windows 装不了"。这些说法有几分真、几分假？实际情况是：OpenClaw 在 Windows、macOS、Linux、云端四大平台上的体验差距确实存在，但远没有传言中那么极端。

这一章，我们不站队、不偏向，用真实数据和社区反馈帮你做判断：每个平台的稳定性如何？功能完整度多少？有哪些独占特性和已知坑点？哪些场景该选哪个平台？读完之后，你就知道自己该把 OpenClaw 装在哪里了。

2.1 四大平台支持概览

在展开各平台的详细分析之前，我们先用一张表建立全局认知。这张表是整个 Part 2 的**速查锚点**——后面每个章节都是在展开解释这张表里的某一行。

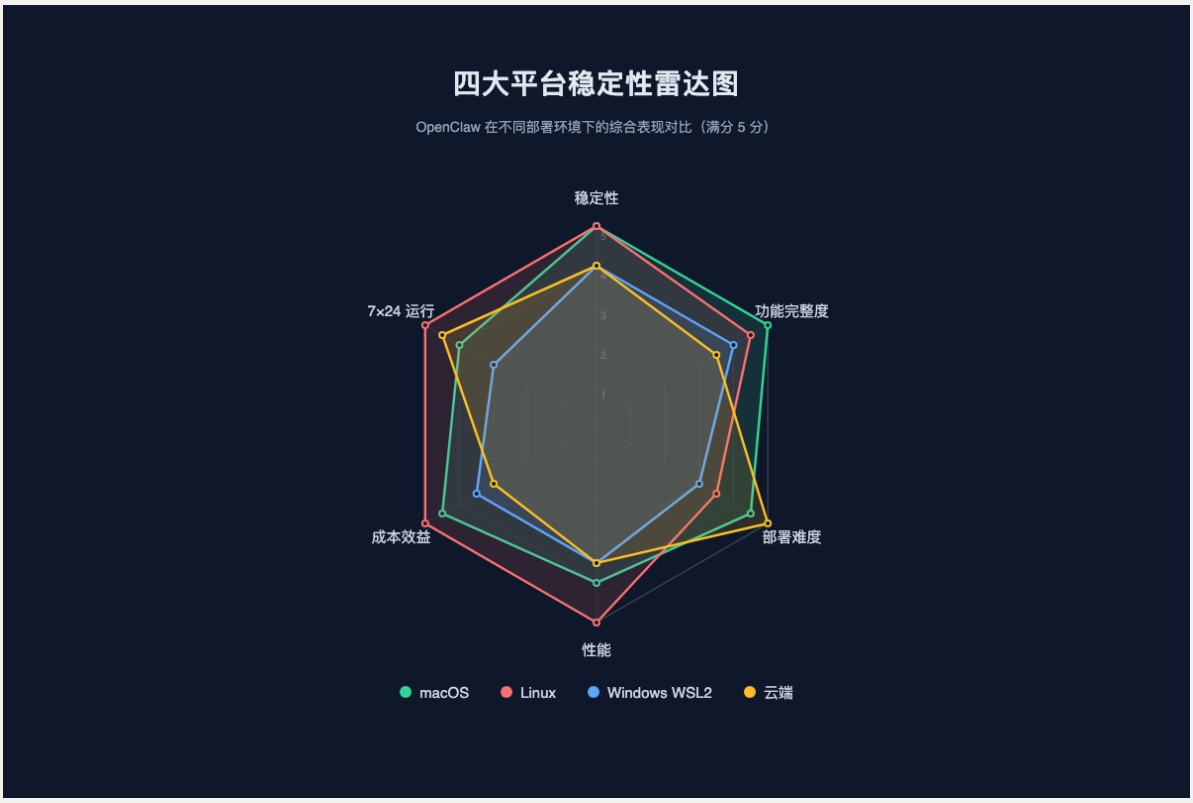
核心指标对比表

维度	WINDOWS (原生)	WINDOWS (WSL2)	MACOS	LINUX	云端
官方定位	❌ 非一等公民	✅ 官方推荐	✅ 一等公民	✅ 原生平台	✅ 一键部署
稳定性	⭐⭐	⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐	⭐⭐⭐⭐
功能完整度	60%	90%	100%	95%	85%
部署难度	难	中等	简单	简单	极简
RAM 占用	不推荐	11-17GB	3-5GB	2-4GB	取决于实例
7×24 运行	❌ 不稳定	⚠️ 更新中断	✅ 可靠	✅ 最佳	✅ 依赖 SLA
本地模型	❌	⚠️ 有限	✅	✅ 最佳	⚠️ 需 GPU 实例
维护成本	高	中	低	低	低（托管）

怎么读这张表？

几个关键维度的含义需要对齐：

- **稳定性**：长时间运行不出问题的概率。五星意味着可以月不重启；两星意味着核心流程可能走不通。
- **功能完整度**：以 macOS 全功能版本为 100% 基准。Linux 缺少 iMessage 等 macOS 独占通道所以是 95%；云端因网络限制和容器环境损失一些能力所以是 85%。
- **部署难度**：从零到运行需要多少步骤和前置知识。macOS 三步搞定（装 Node.js → 装 OpenClaw → 运行引导），Windows 原生则可能在第一步就卡住。
- **RAM 占用**：这是实际测量值。Linux 的 2-4GB 和 WSL2 的 11-17GB 差距之所以这么大，是因为 WSL2 需要运行一个完整的 Linux 内核加上 Hyper-V 虚拟化层，而 Windows 宿主机本身也在消耗内存。



带着这张全局图谱，我们来逐一展开每个平台的细节。

2.2 Windows：能用，但要走对路

> *如果你是 Windows 用户，请先放心——Windows 完全可以运行 OpenClaw。只是需要多走一步，选对路线。*

Windows 的真实现状

OpenClaw 官方在文档中明确表态：

> *"OpenClaw on Windows should be deployed through WSL2 (Ubuntu preferred) rather than native Windows."*

这句话的含义是：**Windows 不是不行，而是要走 WSL2 这条路**。打个比方，在 Windows 上开发 iOS App 需要安装模拟器，在 Windows 上跑 OpenClaw 需要安装 WSL2——本质上都是"借用一个正确的运行环境"。

这并不意味着 Windows 用户是"二等公民"。事实上，通过 WSL2 部署后，你获得的体验和原生 Linux 几乎没有区别。微软在 WSL2 上投入了大量工程资源，它运行的是真实的 **Linux 内核**，不是模拟器。

三种 Windows 方案对比

方案	可行性	稳定性	推荐度	一句话说明
WSL2 + Ubuntu	✅ 可行	⭐⭐⭐⭐	官方推荐	一次设置，长期稳定
原生 **Windows**	⚠️ 部分可行	⭐⭐	不推荐	6 个已确认 Bug，核心流程可能走不通
Docker Desktop	✅ 可行	⭐⭐⭐	替代方案	能用，但资源开销最大

我们的建议很明确：走 **WSL2**。下面解释为什么。

原生 Windows 的已知问题

不回避缺点，是建立信任的最好方式。以下是 GitHub Issues 中已确认的原生 Windows 问题，每一个都有真实用户的复现记录：

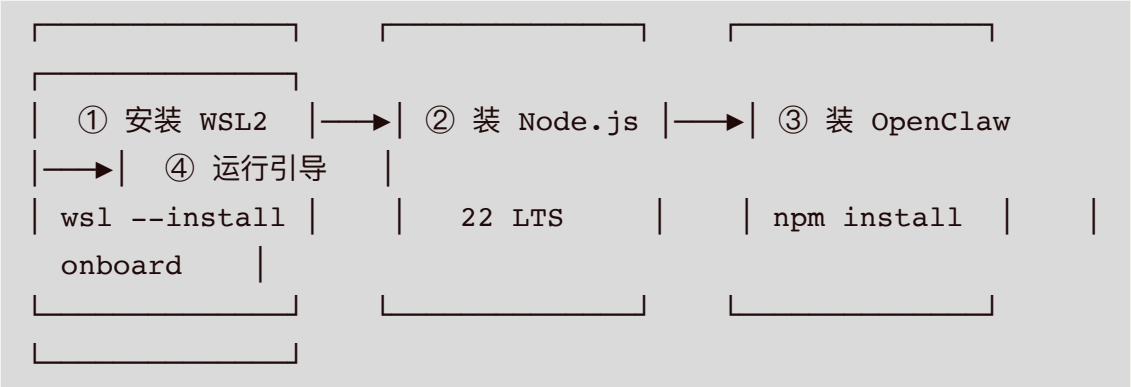
编号	问题	GITHUB ISSUE	严重度
1	安装程序在全新 Win11 上崩溃，PowerShell 立即关闭	#38054	🔴 高
2	Gateway 进程残留导致重启一致性失败	#41804	🔴 高
3	Dashboard 打开显示 "Not Found"	#37036	🔴 高
4	OAuth 登录流程在回调阶段失败	#45036	🟡 中
5	打开配置界面便卡住崩溃	#40836	🟡 中
6	重启脚本弹出多个命令行窗口	#44693	🟢 低

问题 1-3 直接影响安装、启动和管理这三个核心流程。这不是"偶尔有小问题"，而是**核心流程可能走不通**。如果你在原生 Windows 上遇到了类似的问题，不必怀疑是自己操作有误——这就是已知的兼容性缺陷。

WSL2 方案：Windows 用户的正确打开方式

WSL2（Windows Subsystem for Linux 2）是微软官方提供的 Linux 子系统。它不是传统意义上的虚拟机，而是在 Windows 内核中运行一个**轻量级的真实 Linux 内核**。对 OpenClaw 来说，WSL2 环境就是一个标准的 Linux 环境。

核心流程只需 4 步：





安装 WSL2 本身只需要一条 PowerShell 命令（`wsl --install`），之后进入 Ubuntu 环境，体验就和 Linux 一致了。WSL2 还支持 systemd，这意味着你可以用 Linux 标准的方式管理 OpenClaw 服务——开机自启、崩溃恢复，一切和原生 Linux 一样。

WSL2 方案的代价

任何方案都有代价，WSL2 也不例外。了解这些代价，才能做出知情的决策：

代价	具体影响	严重程度
内存开销	WSL2 默认预留 4GB，加上 Windows 宿主机，总占用 11-17GB	🟡 中
跨文件系统性能	WSL 内与 Windows 之间的文件操作慢 5-10 倍	🟢 低（留在 WSL 内操作即可）
IP 地址不稳定	WSL IP 每次重启后变化，端口转发规则需刷新	🟡 中
系统更新中断	Windows 强制更新重启会中断 WSL2 服务	🟡 中（影响 7×24 运行）
虚拟化开销	Hyper-V 虚拟化层增加 5-15% CPU 开销	🟢 低

内存开销是最值得关注的代价。如果你的电脑只有 8GB RAM，WSL2 方案会让系统捉襟见肘。16GB 是底线，32GB 会更从容。

好消息是，跨文件系统性能这个问题几乎不影响 OpenClaw——只要你把所有文件都放在 WSL 内部（`/home/` 路径下），而不是放在 Windows 的 `/mnt/c/` 路径下，就不会触发这个性能瓶颈。

Windows 特有注意事项

问题	说明	应对方式
杀毒软件	Windows Defender 或第三方杀软可能阻止 OpenClaw 的 Node.js 进程	将 Node.js 和 OpenClaw 目录加入白名单
防火墙	默认阻止端口 18789	手动放行入站规则
内存需求	16GB 是底线，8GB 建议放弃 WSL2 方案	内存不足考虑云端部署（2.5 节）
系统更新	强制重启中断 7×24 运行	设置活动时间，延迟更新
路径分隔符	跨 WSL/Windows 路径时注意 `/` vs `\`	尽量在 WSL 内操作

给 Windows 用户的一句话建议

> *电脑内存 >= **16GB**：安装 WSL2，在 Ubuntu 中部署 OpenClaw，体验不输 Linux。*
> *内存不足或不想折腾 **WSL2**：直接看 2.5 节云端部署方案，68 元/年起，比折腾本地更省心。*

2.3 macOS：OpenClaw 的"主场"

为什么 macOS 是最佳体验？

这不是主观偏好，而是工程事实。OpenClaw 的创始人 Peter Steinberger 本人是 Apple 生态开发者，项目从第一天起就在 macOS 上开发和测试。可以说 OpenClaw 是在 **Apple 生态中长大的**。

体现在数据上就是：macOS 功能完整度 100%（其他平台都以它为基准）、独占功能最多、已知 Bug 最少、社区教程最丰富。

macOS 独占功能一览

这张表展示的是只有 **macOS** 才有的功能。其他平台无论怎么配置都无法获得：

独占功能	说明	价值
LaunchAgent 守护	开机自启 + 崩溃自恢复，零配置	7×24 运行的基石，macOS 原生进程管理
菜单栏常驻	原生菜单栏图标，实时显示状态和通知	随时掌握 OpenClaw 运行状态
iMessage 通道	用 Mac 的 iMessage 直接与 AI 对话	Apple 生态用户的专属便利
TCC 权限管理	系统级安全控制（通知/辅助功能/屏幕录制/麦克风）	比手动配置安全策略更优雅
Camera 集成	摄像头直连 OpenClaw	多模态交互基础
Screen Recording	屏幕录制工具集成	视觉感知能力
Canvas 工具	UI 渲染和交互	可视化交互能力

其中 LaunchAgent 值得多说一句。Linux 上的 systemd 虽然也能实现类似的进程守护，但 macOS 的 LaunchAgent 是零配置的——安装时 `openclaw onboard --install-daemon` 一条命令就搞定，不需要手写 service 文件、不需要 `enable-linger`、不需要设置 `XDG_RUNTIME_DIR`。对于不熟悉 Linux 运维的用户，这个差距是显著的。

Apple Silicon 的额外优势

如果你用的是 M 系列芯片的 Mac，还能获得三个硬件层面的加成：

优势	细节
统一内存	M3/M4 Max 最高 128GB 统一内存，CPU 和 GPU 共享。意味着可以直接加载超大本地模型——Ollama 跑 230B MoE 模型（如 MiniMax M2.5）都不在话下
能效比	待机功耗极低，7×24 运行几乎不增加电费。对比 Windows 台式机待机 10-20W，Mac mini 待机只有 2-5W
Metal GPU	Apple 的 Metal 框架可加速部分本地模型推理。虽然不如 NVIDIA CUDA 生态成熟，但对 Ollama 这类集成框架来说已经够用

一个真实的场景：如果你有一台 M4 Max 96GB 的 MacBook Pro，可以同时运行 OpenClaw Gateway + Ollama 加载 Qwen3-32B 模型，全部在本地完成，数据完全不出设备，而且续航依然能撑半天以上。这种体验目前没有其他平台能复制。

macOS 注意事项

虽然 macOS 是最佳平台，但也有几个需要注意的点：

- **避免 iCloud 同步**：配置目录 `~/.openclaw` 千万不要放在 iCloud 同步路径下，否则多设备同步会导致配置冲突
- **不推荐用 Bun 运行时**：官方测试发现 Bun 在 WhatsApp/Telegram 通道上有兼容问题，老老实实用 Node.js
- **exec-approvals.json**：`system.run` 命令的白名单需谨慎配置，它控制着 OpenClaw 能在你电脑上执行哪些系统命令



macOS 的部署流程堪称全平台最简：安装 Node.js → 安装 OpenClaw → 运行引导向导，三步搞定。

2.4 Linux：生产环境的首选

Linux 为什么是生产首选？

macOS 体验最好，但如果要部署一个 7×24 稳定运行的生产级 OpenClaw 实例，Linux 才是正解。原因在底层：

- **Node.js 的原生土壤**：Node.js 本身就是 Linux 生态的产物。`epoll`、`inotify`、信号处理，所有系统调用都是直接的，没有兼容层、没有翻译开销。
- **Docker 零虚拟化开销**：Docker 在 Linux 上共享主机内核，容器运行的 `PID 1` 进程和宿主机上的普通进程没有本质区别。对比 macOS 上的 Docker Desktop 需要跑一个 Linux 虚拟机，Windows 上更是需要 WSL2 + Hyper-V 两层间接。
- **systemd 企业级进程管理**：崩溃自恢复、日志收集（journalctl）、资源限制（cgroups）、依赖管理——Linux 的 init 系统已经在生产环境中验证了二十多年。
- **资源占用最低**：OpenClaw 在 Linux 上实测仅需 2-4GB RAM 即可稳定运行，是所有平台中最低的。

支持的发行版

发行版	推荐度	说明
Ubuntu 22.04 / 24.04	⭐⭐⭐⭐⭐	官方教程主要基于 Ubuntu，遇到问题最容易找到解答
Debian 12 (Bookworm)	⭐⭐⭐⭐⭐	Docker 官方镜像基于 Debian，生产环境首选
CentOS / RHEL	⭐⭐⭐⭐	企业常见环境，需手动安装 Node.js
Alpine	⭐⭐⭐	极致轻量，Docker 镜像仅数十 MB
Arch / Fedora	⭐⭐⭐	社区有安装指南，适合爱折腾的用户

没有特别偏好的话，选 **Ubuntu 22.04**。不是因为它技术上最优，而是因为"出问题时最容易搜到答案"——这在实际使用中的价值远超技术参数的细微差异。

三种部署方式

Linux 的灵活性体现在部署方式的多样性上。三种方式分别对应三种不同的运维水平和使用场景：

部署方式	适合场景	技术门槛	维护成本	安全隔离
npm 全局安装 + systemd	个人服务器、VPS	低	低	一般
Docker 容器	生产环境、多服务共存	中	中	✅ 好
Docker + Kubernetes	企业级、多节点	高	高	✅ 最佳

我们的建议：个人用户直接 npm 安装就够了，不要过度工程化。Docker 方案留给需要安全隔离或多服务共存的场景。Kubernetes 只有真正需要弹性伸缩和多节点高可用时才值得投入。

npm 安装 + systemd 是最直接的路径：三条命令安装、一个 systemd service 文件管理进程、用 `loginctl enable-linger` 确保 SSH 断开后服务继续运行。整个过程不超过 10 分钟。

Docker 方案的核心优势在于**安全隔离**——OpenClaw 运行在一个沙箱容器里，即使 Agent 执行了恶意命令也不会影响宿主机。对于跑在公网上的生产实例，这层隔离值得额外的配置成本。

离线部署方案

100% 纯离线部署是 Linux 的独占优势。在完全断网的内网环境中（政企、军工、金融合规场景），只有 Linux 能可靠地实现全离线部署。

部署思路分两步：

```
┌─ 有网环境（准备阶段）──────────────────────────┐
│  1. 下载 OpenClaw npm 包及全部依赖             │
│  2. 下载 Node.js 安装包                        │
│  3. 下载 Ollama + 模型权重文件（几 GB ~ 几十 GB）│
│  4. 打包为 tar 包或 Docker 镜像                │
└────────────────────────────────────────────────┘

                ↓  USB / 内网传输

┌─ 离线环境（部署阶段）──────────────────────────┐
│  1. 安装 Node.js                               │
│  2. npm install 从本地 tar 包                  │
│  3. 安装 Ollama，导入本地模型权重              │
│  4. 配置 OpenClaw 指向本地 Ollama 端点         │
│  5. 启动 Gateway → 完全离线运行                │
└────────────────────────────────────────────────┘
```



Docker 离线方案更简洁：在有网环境中 `docker save` 导出镜像，传输到离线环境后 `docker load` 导入，一条 `docker compose up -d` 就能启动。

无 systemd 环境（容器/AutoDL）

一些特殊环境（如 AutoDL 的 Docker 容器、部分轻量 VPS）没有 systemd。这种情况下可以用替代方案：

方案	命令	特点
screen	`screen -dmS openclaw bash -c 'openclaw gateway'`	最简单，可随时 attach 查看
nohup	`nohup openclaw gateway > /tmp/openclaw.log 2>&1 &`	标准 Unix 方式
supervisor	配置文件管理	类 systemd 体验，支持自动重启

这不是什么大问题——无 systemd 只是少了"自动管理"的便利，手动管理一样能稳定运行。

2.5 云端部署：零运维的选择

什么样的人应该选云端？

如果你满足以下任一条件，云端部署就是你的最优解：

- 不想折腾本地环境配置
- 需要 7×24 稳定在线（比如挂 Telegram/飞书机器人）
- 电脑内存不足 16GB
- 只想用在线 API，不需要本地模型

国内云厂商一键部署

国内主流云厂商都已提供 OpenClaw 的预装镜像，部署时间可以压缩到 **5 分钟以内**：

厂商	方案	最低价格	推荐配置	亮点
阿里云	轻量应用服务器	**68 元/年**	2C2G 40GB	预装 OpenClaw 镜像，最省心
腾讯云	Lighthouse	99 元/年	2C2G 50GB	控制台可视化配置
百度云	轻量服务器	0.01 元/月	2C2G	"穷鬼套餐"，适合试用体验
火山引擎	云服务器	类似价位	2C4G	有详细踩坑文档

68 元/年是什么概念？一杯奶茶钱就能让 OpenClaw 7×24 在线跑一整年，还不用操心内存不够、系统更新中断、杀毒软件干扰这些本地部署的烦心事。

选哪家没有本质区别，看哪家有活动价即可。如果没有特别偏好，**阿里云 68 元/年的轻量应用服务器**是当前性价比最高的选择——预装镜像省去了所有配置步骤。

云端部署的优势与限制

每个技术选择都有两面性。云端部署省心，但也要清楚它的边界：

维度	优势	限制
部署难度	一键部署，5 分钟上手	需要懂最基础的云服务概念（安全组、SSH）
7×24 运行	天然支持，不受本地开关机影响	依赖云厂商 SLA，免费试用实例可能被回收
安全性	天然网络隔离	Dashboard 暴露在公网需配安全组，建议走 SSH 隧道
本地模型	需另租 GPU 实例（价格较高）	普通 2C2G 实例只能调用在线 API
网络	国内直连各大 API（百炼/DeepSeek/智谱）	无法直连 **HuggingFace / GitHub**，需用国内源
数据安全	取决于云厂商	敏感场景需评估合规性

云 GPU 方案

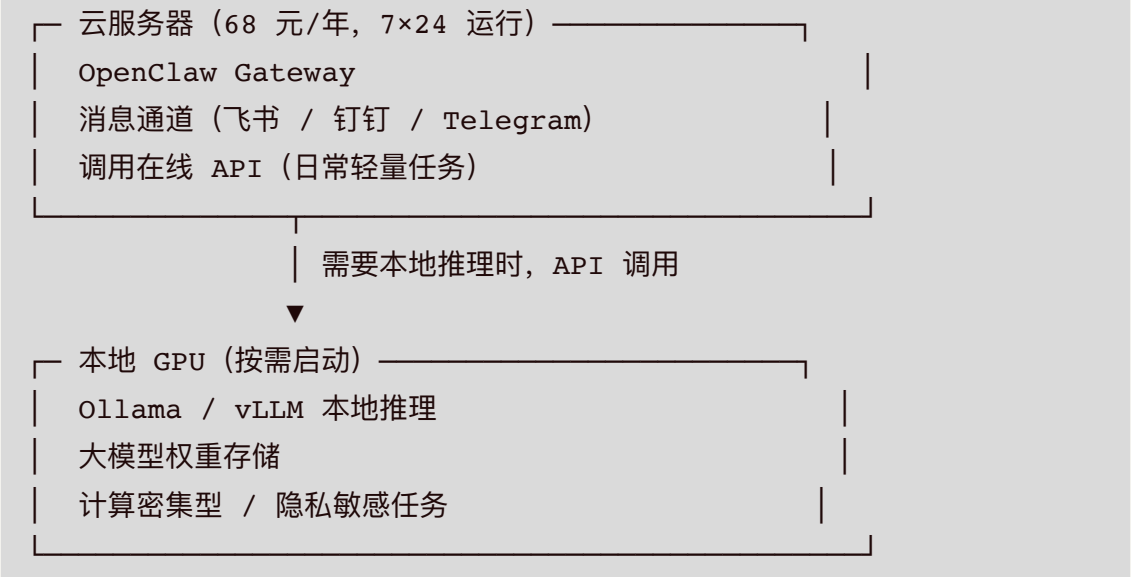
如果你想在云端跑本地模型，就需要 GPU 实例。以下是国内主流的按需 GPU 云平台：

平台	可用 GPU	参考价格	适用场景
AutoDL	A800 80GB / A100 / 3090	¥2-5/小时	大模型推理、按需使用性价比最高
恒源云	A100 / 3090 / 4090	¥1-4/小时	中等模型，竞品替代
阿里云 GPU	V100 / A10G	包月制	企业级、固定需求

GPU 云的核心逻辑是**按需使用**——你不需要 7×24 跑着一块 A800。实际使用中，大多数场景是"白天开机跑推理，晚上关机省钱"。一天用 8 小时，A800 的日成本也就 20-40 元。

云端 + 本地混合方案

真正聪明的做法是**让合适的东西跑在合适的地方**：





这种架构的好处是：Gateway 7×24 在线接收消息（68 元/年的成本可以忽略），本地 GPU 只在需要时启动（节省大量算力费用），隐私敏感的数据留在本地处理（合规友好）。

2.6 本地模型集成概览

OpenClaw 的一大卖点是可以对接本地模型，完全摆脱对在线 API 的依赖。这一节帮你快速了解需要哪些组件、你的硬件能跑多大的模型。

三大推理框架对比

把大模型跑起来需要一个"推理框架"——它负责加载模型权重、处理推理请求、对外暴露 API。目前主流的四个选择：

框架	易用性	并发吞吐	适合场景	OPENCLAW 兼容
Ollama	⭐⭐⭐⭐⭐	低	个人使用，一键安装即跑	✅
vLLM	⭐⭐⭐	极高	生产部署，多并发场景	✅（推荐）
LM Studio	⭐⭐⭐⭐	低	桌面用户，GUI 操作	✅
llama.cpp	⭐⭐	中	极致性能、自定义量化、边缘设备	需适配

一句话选择指南：个人用 Ollama，生产用 vLLM。

展开说几句：Ollama 的核心价值是简单——`ollama run qwen3:32b` 一条命令就能下载模型并启动推理，它自动管理模型权重、量化格式、GPU 调度，对新手极其友好。代价是并发能力有限，不适合多人同时使用。

vLLM 则是另一个方向——它使用 PagedAttention 技术，能在相同 GPU 显存下支持比 Ollama 高 3-5 倍的并发吞吐。配置门槛稍高，但如果你需要在一台 A800 上同时服务多个 Agent，vLLM 是唯一合理的选择。

所有这些框架都通过 **OpenAI 兼容 API** 与 OpenClaw 对接，配置方式完全相同——只需要在 OpenClaw 配置文件中填写 `base_url` 和 `model` 即可。

硬件 ↔ 模型速查表

"我的显卡能跑多大的模型？"——这可能是最高频的问题。以下速查表给出直接的答案：

硬件	显存 / 内存	可运行模型	典型代表
RTX 3060	12GB	7-8B 模型	Qwen 2.5 Coder 7B
RTX 3090 / 4090	24GB	32B 量化模型	Qwen3-32B (Q4)、Mistral Small 3.1
A800 / A100	80GB	70B+ 完整精度	Qwen3-72B、Llama 4 Scout
Mac M3/M4 Max	128GB 统一内存	230B MoE 模型	MiniMax M2.5 (230B)
无 **GPU**（纯 **CPU**）	依赖 RAM	7B 量化模型（速度较慢）	Qwen 2.5 7B (Q4)

几个关键判断点：

- **24GB 是甜蜜点**：RTX 4090 的 24GB 显存可以舒服地跑 32B 量化模型，这个规模的模型在代码和通用任务上已经相当能打
- **80GB 解锁 70B+**：如果你需要跑 70B 级别的大模型，至少需要 A100/A800 80GB。这也是云 GPU 方案（如 AutoDL）最常用的场景
- **Mac 统一内存是隐藏大杀器**：128GB 统一内存意味着 CPU 和 GPU 共享这些内存，Ollama 可以直接利用全部内存加载模型，不受 "显存" 限制

> *温馨提示：以上是 BF16/FP16 全精度下的参考值。如果使用 4-bit 量化（Q4_K_M），模型的显存需求大约降为全精度的 1/3 ~ 1/4，但推理质量会有一定损失。*

2.7 平台选择推荐与总结

前面六节拆解了每个平台的细节，这一节把它们拼回一张完整的决策地图。不管你是什么背景、什么需求，在这里都能找到明确的推荐。

按用户画像选平台

你是谁	推荐平台	理由	月成本参考
Mac 用户	macOS 原生安装	体验最佳、功能最全、零额外配置	电费可忽略
Mac + 本地大模型	macOS + Ollama	Apple Silicon 统一内存优势大，数据不出设备	电费可忽略
Windows 用户	WSL2 + Ubuntu	官方推荐路线，避免原生 6 个已知 Bug	电费可忽略
Linux 服务器运维	Linux + Docker	生产级稳定、安全隔离、资源占用最低	VPS ¥5-50/月
零运维新手	阿里云/腾讯云一键部署	68 元/年起，5 分钟上手，最省心	¥6-8/月
隐私优先	Linux + Ollama 离线	数据完全不出本机，满足合规要求	硬件一次性投入
GPU 大模型玩家	AutoDL/恒源云 + vLLM	GPU 按需租用，成本可控	按用量 ¥20-150/月
企业/团队	云服务器 + Docker + K8s	弹性伸缩、多节点、SLA 保障	取决于规模

如果你还在犹豫，给一个最简单的判断流程：

```
你有 Mac 吗?
  ├── 是 → macOS 原生安装，结束
  └── 否 → 你愿意折腾 WSL2 吗?
             ├── 是 → WSL2 + Ubuntu，结束
             └── 否 → 阿里云 68 元/年一键部署，结束
```

平台独占功能汇总

最后用一张表梳理各平台的独占能力，帮助你理解选择某个平台时"得到了什么"和"放弃了什么"：

功能	MACOS	LINUX	WINDOWS WSL2	云端
LaunchAgent 守护	✅	—	—	—
菜单栏 App	✅	—	—	—
iMessage 通道	✅	—	—	—
TCC 权限管理	✅	—	—	—
systemd 服务	—	✅	✅	部分
Docker 零开销	—	✅	—	✅
一键部署镜像	—	—	—	✅
GPU 直通推理	✅ (Metal)	✅ (CUDA)	⚠️ 有限	✅ (CUDA)
离线部署	⚠️ 可行	✅ 最佳	⚠️ 可行	❌

几个值得关注的发现：

- **macOS 独占 4 项**：LaunchAgent、菜单栏、iMessage、TCC——这些都是 Apple 系统 API 深度集成的结果，其他平台从技术上就无法实现
- **Linux 的核心优势不在独占功能，而在"做什么都最好"**：Docker 零开销、systemd、GPU CUDA 直通、离线部署——这些其他平台也能做，但 Linux 上做得最干净、最高效
- **云端的独占价值是"一键部署"和"7×24 免运维"**：如果你不需要这两点，本地部署在各方面都更有优势
- **Windows WSL2 没有独占功能**：它的价值是"让 Windows 用户也能用上 Linux 的能力"，本质上是在 Windows 上借用 Linux 的优势